

Expresiones

Material de lectura sobre el tema: Complementar lo presentado aquí con:

- Sebesta: Capítulo 7.
- Scott: Capítulo 6 (sección 6.1).

La carrera de la abstracción: expresiones

Lo siguiente en incorporarse a nivel abstracción fueron las **expresiones** y nuevamente un lenguaje moderno sin expresiones es casi inconcebible. Aún así, hay detalles de diseño que es interesante conocer y analizar.

Para comenzar, las expresiones están dentro de lo que se conoce como el *control de flujo del programa*. El control de flujo es uno de los temas centrales de un lenguaje de programación, la manera como se incorpore en el lenguaje determina el orden de ejecución de un programa y por lo tanto tiene un impacto importante sobre la semántica.

Los mecanismos del lenguaje utilizados para especificar orden de ejecución pueden ordenarse en las siguientes categorías:

- a) Secuencia
- b) Selección
- c) Iteración
- d) Abstracción procedural
- e) Recursión
- f) Concurrencia
- g) Manejo de excepciones o especulación
- h) No determinismo

Muchas de estas formas de control de flujo requieren del uso de expresiones. Las expresiones son una forma fundamental de especificar computación en un lenguaje de programación. Además de la forma de las mismas (sintaxis: BNF), su significado (semántica) es crucial. **La semántica de las expresiones está gobernada por el orden de evaluación.** Es necesario ser conscientes de los órdenes de evaluación de operandos y operadores para interpretar adecuadamente las expresiones en el lenguaje.

Una expresión consiste generalmente de un objeto simple (constante literal, constante nombrada o variable), o un operador o una función aplicada a una colección de operandos o argumentos (cada uno de los cuales define a su vez una expresión).

Luego, **las expresiones constan de: operadores, operandos, paréntesis y llamadas a abstracciones funcionales.** Si bien hay varias maneras de establecer la sintaxis de las expresiones (notación prefija, infija, postfija), la mayoría de los lenguajes imperativos utilizan notación infija para los operadores binarios y prefija para los unarios. Claramente no es la mejor manera para la máquina, pero sí para el programador. El funcionamiento interno de la computadora se lleva mucho mejor con las otras notaciones, es por eso que para algunos lenguajes los operadores infijos se piensan solo como “azúcar sintáctico” (“syntactic sugar”) para conseguir un aspecto más normal. Así, por ejemplo, $a+b$ es internamente $+(a,b)$.

Consideraciones de diseño para las expresiones:

- ✧ Reglas de precedencia de operadores.
- ✧ Reglas de asociatividad de operadores.
- ✧ Orden de evaluación de los operandos.
- ✧ Evaluación de efectos colaterales de un operador.
- ✧ Sobrecarga de un operador.
- ✧ Modo mixto de expresiones.
- ✧ Errores en las expresiones.

Con respecto a las reglas de precedencia y asociatividad, aunque el lenguaje puede definir las reglas que desee, lo habitual es tomar las establecidas por la lógica y la matemática. De esta manera se evita la posibilidad de errores de interpretación.

La asignación

En general las expresiones son utilizadas en el contexto de un constructor de control o de una sentencia. La sentencia que se viene a la mente en primera instancia es la sentencia de **asignación**. La interpretación de la misma es bastante diferente de acuerdo al paradigma y a la concepción de la misma ya que no siempre se interpreta como una sentencia. Así, en el paradigma funcional, como el cómputo consiste en evaluar expresiones, las expresiones resultan ser los bloques básicos del lenguaje por lo que la asignación no tiene el sentido convencional. Lo más parecido a una asignación sería la declaración de un nombre local. Tengan en cuenta que en el paradigma funcional, el concepto de variable es el concepto matemático de variable (las variables no son abstracciones de celdas de memoria, sino la abstracción de un valor). Por su parte, en el paradigma imperativo el cómputo consiste en una serie ordenada de cambios (de los valores de las variables en memoria), y las asignaciones resultan el medio principal para realizar esos cambios.

La sintaxis general en los paradigmas convencionales tiene la siguiente forma:

`<variable destino> <operador de asignación> <expresión>`

El operador de asignación varía de lenguaje de programación en lenguaje de programación, las versiones más conocidas son “:=” (provenientes de la familia de Algol) y “=” (de la familia de C).

A simple vista, la asignación parece una operación directa. A nivel implementación, sin embargo, se producen diferencias sutiles que pueden dar lugar a diferentes tipos de asignación. Estas diferencias a simple vista pueden resultar invisibles (no afectan el comportamiento en programas simples).

En C, podemos escribir asignaciones como:

`d = a` `a = b + c`

En la primera asignación, el lado derecho hace referencia al valor de `a`; mientras que, en la segunda asignación el lado izquierdo hace referencia a la locación de `a`.

La semántica de la operación de asignación puede pensarse como:

- ✓ Copia de la referencia.
- ✓ Dereferenciamiento implícito.

Los pasos que definen la operación de asignación serían:

- 1) Computar el valor del lado izquierdo (l-value: son expresiones que denotan locaciones).
- 2) Computar el valor de la expresión del lado derecho (r-value: se trata de expresiones que denotan valores).
- 3) Asignar el valor computado del lado derecho al computado como objeto de dato del lado izquierdo.
- 4) Retornar el valor computado como valor del lado derecho como resultado de la asignación.

El último paso solo tiene lugar si la asignación se piensa como una expresión y no como una instrucción o sentencia.

Lenguajes como C toman esta acepción de las asignaciones y por eso permiten asignaciones como “`a = b = c`”: el resultado de la asignación `b=c` se asigna a `a`. Si la asignación no tuviera valor de retorno no podría permitirse. En este tipo de caso la asignación tiene dos resultados, el cambio de valor de la variable que está del lado izquierdo y el retorno del valor de la expresión de la derecha.

Algunos lenguajes también proveen operaciones de asignación combinada, la idea es proveer facilidades de escritura en asignaciones triviales. Así, por ejemplo, en lenguajes como C una asignación trivial como `a = a+1` puede escribirse como `a++`. Este tipo de agregado sintáctico no modifica la expresividad del lenguaje ya que el lenguaje ya tenía un mecanismo para expresar lo deseado, simplemente mejora la facilidad de escritura. Se puede notar que el abuso de este tipo de notación combinado con el tener la asignación como expresión puede llevar a expresiones muy difíciles de leer.

La asignación no siempre ocurre de manera explícita, hay circunstancias en las que tiene lugar de manera implícita. Generalmente esto sucede en la inicialización de variables en las declaraciones y la asociación de parámetros actuales y formales en una invocación a una unidad.

Ordenamiento

Las **reglas de precedencia** establecen el orden en el que los operadores infijos de una expresión se aplican, pero no determinan el orden de evaluación total de la expresión, ya que si hay dos operadores con la misma precedencia en la expresión se pueden resolver en cualquier orden. Por otro lado, las **reglas de asociatividad** definen cómo se resuelve el orden de evaluación frente a operadores del mismo nivel de precedencia. La regla típica adoptada por los lenguajes de programación es la de evaluación de izquierda a derecha (ya que coincide con el procesamiento del archivo que contiene el código al momento de reconocer y traducir).

El constructor de un lenguaje de programación produce un **efecto colateral** si influencia al resto de la computación de otra manera que no sea a través de su retorno.

Si ninguno de los operandos u operadores tiene efectos colaterales, entonces el orden de evaluación es irrelevante, pero en computación tener bien establecido el orden es fundamental por dos motivos:

- ✓ Precisamente por la posibilidad de que algún operador u operando tenga o genere efectos colaterales.
- ✓ Optimización de código y eficiencia en la implementación del reconocedor/traductor.

Contrariamente a lo que sucede en matemática, la evaluación de una expresión en un lenguaje de programación puede generar resultados diferentes a causa de los efectos colaterales. Una expresión tan simple como $a+f(b)$ puede dar resultados diferentes si la función f genera un efecto colateral sobre el valor de a . La variable a no tiene el mismo valor antes, que después de la evaluación de f .

Evaluación de las expresiones

Hay básicamente dos caminos a seguir a la hora de decidir cómo evaluar las expresiones:

- a. Evaluación Estricta.
- b. Evaluación No Estricta.

Evaluación estricta, también conocida como evaluación de orden aplicativo, ya que se evalúa de izquierda a derecha, o evaluación ansiosa. En este tipo de evaluación los argumentos de una función o los operandos de la expresión son siempre completamente evaluados antes de la aplicación de la función u operador. Las expresiones se evalúan al momento de establecer la ligadura con una variable.

Los lenguajes imperativos la utilizan casi siempre (debido a que el orden de ejecución está definido implícitamente por la organización del código).

Entre las principales ventajas de esta forma de evaluación se destacan: la eliminación de la necesidad de planificar como resolver la expresión y el hecho de que el programador determina la semántica de la expresión a través de la forma en que la expresa (lo que es importante cuando hay efectos colaterales involucrados). Como siempre, también hay desventajas. Entre ellas se destacan el hecho de que fuerza la evaluación de expresiones que pueden no ser necesarias en tiempo de ejecución y que fuerza al programador a organizar el código fuente para garantizar una ejecución óptima.

Evaluación no estricta, también conocida como evaluación de orden normal. En este caso, los argumentos de una función o los operandos de la expresión **no son evaluados a menos que sean utilizados** en el cuerpo de la función o sean necesarios para determinar el valor de la expresión. Sus principales características son: el hecho de que la evaluación se reduce por la izquierda y el hecho de que las funciones se aplican antes de evaluar sus argumentos.

Este tipo de evaluación se puede especializar en distintas versiones:

- ✧ *Evaluación perezosa*: agrega la memorización de los argumentos evaluados.
- ✧ *Evaluación corto-circuito* (en expresiones booleanas): se retorna el valor de la expresión tan pronto como se pueda determinar el valor de verdad sin ambigüedad.

En un principio esta forma de evaluación parecería ser más eficiente ya que evita el cálculo de partes de la expresión que no son necesarios para determinar el valor de la misma. Sin embargo, ese ahorro de cómputo se ve empañado por la sobrecarga que implica la planificación de la evaluación de la expresión y que cuando se combina con pasaje de parámetros (como veremos más adelante en el curso) implica sostener ambientes de referenciamiento. Como

consecuencia de esto, se gasta más de lo que se ahorra y es por eso que en general los lenguajes dejan esta forma de evaluación o cualquiera de sus versiones solo para operadores particulares.

Un ejemplo donde esta forma de evaluación puede apreciarse es en la siguiente expresión condicional en C:

$$a = 1 ? 0 : b/a$$

Si quisiéramos evaluar esta expresión condicional de manera ansiosa tendríamos un error en ejecución, ya que la evaluación ansiosa evalúa las tres expresiones involucradas: la condición, la primera expresión (que es trivial en este caso por tratarse de un valor) y la segunda expresión (la división).

Si lo trasladamos a una expresión condicional en un lenguaje funcional como ML:

$$\text{fun } g(x,y) = \text{if } x < 2 \text{ then } y+3 \text{ else } g(x-1,y)+1$$

En este caso el problema de evaluación que surge es que hay que procesar la llamada recursiva todas las veces, por lo que el cálculo no termina nunca.

Estos dos operadores particulares tienen evaluación perezosa en sus respectivos lenguajes de origen, aunque en general el resto de la evaluación sea ansiosa.